



**SWAMINARAYAN
UNIVERSITY**

THE RESERVOIR OF ALL KNOWLEDGE

**FACULTY OF ENGINEERING
DIPLOMA ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING
SCRIPTING LANGUAGE PYTHON
(DE103301)**

Unit 1: Fundamentals Of Python

Introduction to Python

- Python is a widely used general-purpose, high level programming language.
- Python is interpreted object-oriented language.
- It is open source software and easily available to use.

- It enables the straight and easy programming for both small and large applications. It mainly emphasizes on code reusability, readability and using white space.
- It has features like high-level built-in data structures, dynamic typing, and binding, which makes it attractive for rapid application development.
- Python is a programming language that lets you work quickly and integrate systems more efficiently.

History of Python

- It was initially designed by **Guido van Rossum in 1991** and developed by Python Software Foundation.
- There are two major Python versions- **Python 2 and Python 3.**
- On 16 October 2000, Python 2.0 was released with many new features.
- On 3rd December 2008, Python 3.0 was released with more testing and includes new features.

Below are some facts about Python Programming Language:

1. Python is currently the most widely used multi-purpose, high-level programming language.
2. Python allows programming in Object-Oriented and Procedural paradigms.
3. Python programs generally are smaller than other programming languages like Java. Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.
4. Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber... etc.
5. The biggest strength of Python is huge collection of standard library which can be used for the following:
 - Machine Learning
 - GUI Applications (like Kivy, Tkinter, PyQt etc.)
 - Web frameworks like Django (used by YouTube, Instagram, Dropbox)
 - Image processing (like OpenCV, Pillow)
 - Web scraping (like Scrapy, BeautifulSoup, Selenium)
 - Test frameworks
 - Multimedia
 - Scientific computing
 - Text processing and many more..

Python Applications

1) Web Applications: We can use Python to develop web applications. It provides libraries to handle internet protocols such as **HTML** and XML, JSON, Email processing, request, etc. One of Python web-framework named Django is used on Instagram. Python provides many useful frameworks, and these are given below:

O Django and Pyramid framework(Use for heavy applications)
O Flask and Bottle (Micro-framework)

2) Desktop GUI Applications: The GUI stands for the Graphical User Interface, which provides a smooth interaction to any application. Python provides a Tk GUI library to develop a user interface.

3) Console-based Application: Console-based applications run from the command-line or shell. These applications are computer programs which are used to execute commands. Python can develop this kind of application very effectively.

4) Software Development: Python is useful for the software development process. It works as a support language and can be used to build control and management, testing, etc.

5) Scientific and Numeric: Python language is the most suitable language for Artificial intelligence or machine learning. It consists of many scientific and mathematical libraries, which makes it easy to solve complex calculations.

6) Business Applications: E-commerce and **ERP** are an example of a business application. This kind of application requires extensibility, scalability and readability, and Python provides all these features.

7) Audio or Video-based Applications : Python is flexible to perform multiple tasks and can be used to create multimedia applications. Some multimedia applications which are made by using Python are TimPlayer, cplay, etc.

8) 3D CAD Applications: The CAD (Computer-aided design) is used to design engineering related architecture. It is used to develop the 3D representation of a part of a system. Python can create a 3D CAD application

Features in Python

There are many features in Python, some of which are discussed below as follows:

1. Free and Open Source

Python language is freely available at the official website and you can download it from the given download link below click on the **Download Python** keyword. Download Python Since it is open-source; this means that source code is also available to the public. So you can download it, use it as well as share it.

2. Easy to code

Python is a high-level programming language. Python is very easy to learn the language as compared to other languages like C, C#, Javascript, Java, etc. It is very easy to code in the Python language and anybody can learn Python basics in a few hours or days. It is also a developer-friendly language.

3. Easy to Read

As you will see, learning Python is quite simple. As was already established, Python's syntax is really straightforward. The code block is defined by the indentations rather than by semicolons or brackets.

4. Object-Oriented Language

One of the key features of Python is Object-Oriented programming. Python supports object-oriented language and concepts of classes, object encapsulation, etc.

5. GUI Programming Support

Graphical User interfaces can be made using a module such as PyQt5, PyQt4, wxPython, or Tk in python. PyQt5 is the most popular option for creating graphical apps with Python.

6. High-Level Language

Python is a high-level language. When we write programs in Python, we do not need to remember the system architecture, nor do we need to manage the memory.

7. Extensible feature

Python is an **Extensible** language. We can write some Python code into C or C++ language and also we can compile that code in C/C++ language.

8. Easy to Debug

Excellent information for mistake tracing. You will be able to quickly identify and correct the majority of your program's issues once you understand how to interpret Python's error traces. Simply by glancing at the code, you can determine what it is designed to perform.

9. Python is a Portable language

Python language is also a portable language. For example, if we have Python code for windows and if we want to run this code on other platforms such as Linux, Unix, and Mac then we do not need to change it, we can run this code on any platform.

10. Python is an integrated language

Python is also an integrated language because we can easily integrate Python with other languages like C, C++, etc.

11. Interpreted Language:

Python is an Interpreted Language because Python code is executed line by line at a time. Like other languages C, C++, Java, etc. there is no need to compile Python code this makes it easier to debug our code. The source code of Python is converted into an immediate form called **bytecode**.

12. Large Standard Library

Python has a large standard library that provides a rich set of modules and functions so you do not have to write your own code for every single thing. There are many libraries present in Python such as regular expressions, unit-testing, web browsers, etc.

13. Dynamically Typed Language

Python is a dynamically-typed language. That means the type (for example- int, double, long, etc.) for a variable is decided at run time not in advance because of this feature we don't need to specify the type of variable.

14. Frontend and backend development

With a new project py script, you can run and write Python codes in HTML with the help of some simple tags <py-script>, <py-env>, etc. This will help you do frontend development work in Python like javascript. Backend is the strong forte of Python it's extensively used for this work cause of its frameworks like Django and Flask.

15. Allocating Memory Dynamically

In Python, the variable data type does not need to be specified. The memory is automatically allocated to a variable at runtime when it is given a value. Developers do not need to write `int y = 18` if the integer value 15 is set to y. You may just type `y=18`.

Python Keywords and Identifiers

Keywords: are some predefined and reserved words in python that have special meanings. Keywords are used to define the syntax of the coding. The keyword cannot be used as an identifier, function, and variable name. All the keywords in python are written in lower case except True and False. There are 33 keywords in Python 3.7

Identifier is a name used to identify a variable, function, class, module, etc. The identifier is a combination of character digits and underscore. The identifier should start with a character or Underscore then use a digit. The characters are A-Z or a-z, an Underscore (`_`), and digit (0-9). We should not use special characters (`#`, `@`, `$`, `%`, `!`) in identifiers.

Examples of valid identifiers:

- `var1`
- `_var1`
- `_1_var`
- `var_1`

Examples of invalid identifiers

- `!var1`
- `1var`
- `1_var`
- `var#1`

Total Python keywords

Keywords	Description
and	This is a logical operator it returns true if both the operands are true else return false.
Or	This is also a logical operator it returns true if anyone operand is true else return false.
not	This is again a logical operator it returns True if the operand is false else return false.
if	This is used to make a conditional statement.
elif	Elif is a condition statement used with an if statement the elif statement is executed if the previous conditions were not true
else	Else is used with if and elif conditional statement the else block is executed if the given condition is not true.
for	This is created for a loop.
while	This keyword is used to create a while loop.
break	This is used to terminate the loop.
as	This is used to create an alternative.
def	It helps us to define functions.

Keywords	Description
lambda	It is used to define the anonymous function.
pass	This is a null statement which means it will do nothing.
return	It will return a value and exit the function.
True	This is a boolean value.
False	This is also a boolean value.
try	It makes a try-except statement.
with	The with keyword is used to simplify exception handling.
assert	This function is used for debugging purposes. Usually used to check the correctness of code
class	It helps us to define a class.
continue	It continues to the next iteration of a loop
del	It deletes a reference to an object.
except	Used with exceptions, what to do when an exception occurs
finally	Finally is use with exceptions, a block of code that will be executed no matter if there is an exception or not.
from	The form is used to import specific parts of any module.
global	This declares a global variable.
import	This is used to import a module.
in	It's used to check if a value is present in a list, tuple, etc, or not.

Keywords

Description

is	This is used to check if the two variables are equal or not.
None	This is a special constant used to denote a null value or avoid. It's important to remember, 0, any empty container(e.g empty list) do not compute to None
nonlocal	It's declared a non-local variable.
raise	This raises an exception
yield	Its ends a function and returns a generator.

Variable of python

Python Variable is containers which store values. Python is not “statically typed”. We do not need to declare variables before using them or declare their type. A variable is created the moment we first assign a value to it. A Python variable is a name given to a memory location. It is the basic unit of storage in a program.

- *The value stored in a variable can be changed during program execution.*
- *A Python Variable is only a name given to a memory location, all the operations done on the variable effects that memory location.*
-

Rules for creating variables in Python

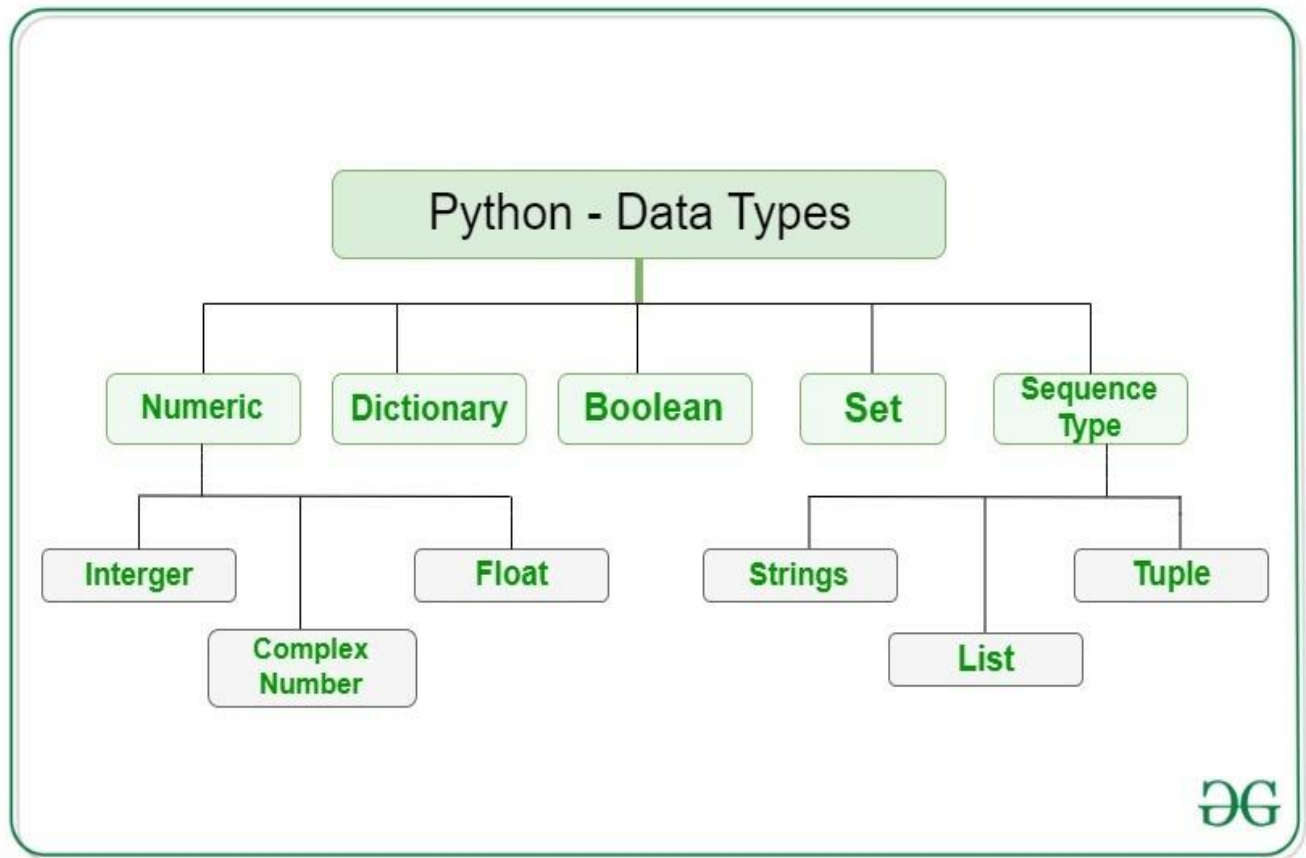
- A variable name must start with a letter or the underscore character.
- A variable name cannot start with a number.
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _).
- Variable names are case-sensitive (name, Name and NAME are three different variables).
- The reserved words (keywords) cannot be used naming the variable.

Python Data Types

Data types are the classification or categorization of data items. It represents the kind of value that tells what operations can be performed on a particular data. Since everything is an object in Python programming, data types are actually classes and variables are instance (object) of these classes. Following are the standard or built-in data type of Python:

- Numeric
- Sequence Type
- Boolean

- Set
- Dictionary



Numeric

In Python, numeric data type represents the data which has numeric value. Numeric value can be integer, floating number or even complex numbers. These values are defined as int, float and complex class in Python.

- **Integers** – This value is represented by int class. It contains positive or negative whole numbers (without fraction or decimal). In Python there is no limit to how long an integer value can be.
- **Float** – This value is represented by float class. It is a real number with floating point representation. It is specified by a decimal point. Optionally, the character e or E followed by a positive or negative integer may be appended to specify scientific notation.
- **Complex Numbers** – Complex number is represented by complex class. It is specified as *(real part) + (imaginary part)j*. For example – 2+3j

a) Integers are the whole numbers consisting of + or – sign
e.g. age=19

salary=20000

b) Floating Point: Numbers with fractions or decimal floating point numbers will consist of sign (+,-) and
e.g. temperature = -21.9, growth_rate = 0.98333

c) Complex: Complex number is made up of two floating point values, one each for real and imaginary part. For accessing different parts of a variable x we will use x.real and x.imag. Imaginary part of the number is represented by j instead of i, so 1+0j denotes zero imaginary part.

Example

```
>>> x = 1+0j
>>> print x.real, x.imag
1.0 0.0
```

2. Boolean: Object of Boolean type may have one of two values, True or False: >>>

```
<class 'bool'>
>>> type(False)
<class 'bool'>
```

3. **Sequence:** A sequence is an ordered collection of items, indexed by positive integers. Three types of sequence data type available in Python are Strings, Lists & Tuples.

a) **String:** is an ordered sequence of letters/characters. They are enclosed in single quotes (') or double quotes (").

Example

```
>>> a = 'CE'
>>> a = "CE"
```

b) **Lists:** List is also a sequence of values of any type. Values in the list are called elements/items. The items of list are accessed with the help of index (index starts from 0). List is enclosed in square brackets.

Example

```
Student = ["Dharmesh", 567, "CS"]
```

c) **Tuples:** Tuples are a sequence of values of any type, and are indexed by integers. They are immutable i.e. we cannot change the value of items Of tuples . Tuples are enclosed in square bracket, Student = ("Dharmesh", 567, "CS")

4. Sets

Set is an unordered collection of values, of any type, with no duplicate entry. Sets are immutable. Example

```
s = set ([1,2,3,4])
```

5. **Dictionaries:** Dictionaries store a key - value pairs, which are enclosed in curly brackets.

Example

```
d = {1:'a', 2:'b', 3:'c'}
```

Type casting in python

Python defines type conversion functions to directly convert one data type to another which is useful in day-to-day and competitive programming. This article is aimed at providing information about certain conversion functions.

There are two types of Type Conversion in Python:

1. Implicit Type Conversion

2. Explicit Type Conversion

Let's discuss them in detail.

Implicit Type Conversion

In Implicit type conversion of data types in Python, the Python interpreter automatically converts one data type to another without any user involvement..

Explicit Type Conversion

In Explicit Type Conversion in Python, the data type is manually changed by the user as per their requirement. With explicit type conversion, there is a risk of data loss since we are forcing an expression to be changed in some specific data type

1. **int(a, base):** This function converts **any data type to integer**. 'Base' specifies the **base in which string is** if the data type is a string.

2. **float():** This function is used to convert **any data type to a floating-point number**.

3. **ord ()** : This function is used to convert a **character to integer**.

4. **hex ()** : This function is to convert **integer to hexadecimal string**.

5. **oct()** : This function is to convert **integer to octal string**.

Input and Output in Python

In this article, we will see different ways in which first we can take input from users and second show output to them.

How to Take Input from User in Python

Sometimes a developer might want to take user input at some point in the program. To do this Python provides an `input()` function.

Syntax:

```
input('prompt')
```

Where `prompt` is an optional string that is displayed on the screen at the time of taking input.

Example 1: Python get user input with a message

- Python3

```
name = input("Enter your name: ")
```

```
print("Hello, " + name)
```

```
print(type(name))
```

Output:

```
Enter your name: GFG
```

```
Hello, GFG
```

```
<class 'str'>
```

Note: Python takes all the input as a string input by default. To convert it to any other data type we have to convert the input explicitly. For example, to convert the input to `int` or `float` we have to use the [int\(\)](#) and [float\(\)](#) method respectively.

Example 2: Integer input in Python

- Python3

```
# Taking input from the user as integer
```

```
num = int(input("Enter a number: "))
```

```
add = num + 1
```

```
# Output
```

```
printint(add)
```

Output:

```
Enter a number: 25
```

```
26
```

How to take Multiple Inputs in Python :

We can take multiple inputs of the same data type at a time in python, using map() method in python.

- Python3

```
a, b, c = map(int, input("Enter the Numbers : ").split())
```

```
print("The Numbers are : ",end = " ")
```

```
print(a, b, c)
```

Output:

Enter the Numbers: 2 3 4

The Numbers are: 2 3 4

How take inputs for the Sequence Data Types like List, Set, Tuple, etc.

In the case of List and Set the input can be taken from the user in two ways.

1. Taking List/Set elements one by one by using the append()/add() methods.
2. Using map () and list () / set () methods.

Taking List/Set elements one by one

Take the elements of the List/Set one by one and use the append() method in the case of List, and add() method in the case of a Set, to add the elements to the List / Set.

- Python3

```
List = list()
```

```
Set = set()
```

```
l = int(input("Enter the size of the List : "))
```

```
s = int(input("Enter the size of the Set : "))
```

```
print("Enter the List elements : ")
```

```
for i in range(0, l):
```

```
    List.append(int(input()))
```

```
print("Enter the Set elements : ")
```

```
for i in range(0, s):
```

```
    Set.add(int(input()))
```

```
print(List)
```

```
print(Set)
```

Output :

Enter the size of the List : 4

Enter the size of the Set : 3

Enter the List elements :

9

0

1

3

Enter the Set elements :

2

9

1

[9, 0, 1, 3]

{9, 2, 1}

Using map () and list () / set () Methods

- Python3

```
List = list(map(int, input("Enter List elements : ").split()))
```

```
Set = set(map(int, input("Enter the Set elements :").split()))
```

```
print(List)
```

```
print(Set)
```

Output :

Enter List elements : 3 4 8 9 0 11

Enter the Set elements :34 88 230 234 123

[3, 4, 8, 9, 0, 11]

{34, 230, 234, 88, 123}

Taking Input for Tuple

We know that tuples are immutable; there are no methods available to add elements to tuples. To add a new element to a tuple, first type cast the tuple to the list, later appends the element to the list, and again type cast list back to a tuple.

- Python3

```
T = (2, 3, 4, 5, 6)
```

```
print("Tuple before adding new element")
```

```
print(T)
```

```
L = list(T)
```

```
L.append(int(input("Enter the new element : ")))
```

```
T = tuple(L)
```

```
print("Tuple After adding the new element")
```

```
print(T)
```

Output:

Tuple before adding new element

(2, 3, 4, 5, 6)

Enter the new element: 35

Tuple After adding the new element

(2, 3, 4, 5, 6, 35)

How to Display Output in Python

Python provides the [print\(\)](#) function to display output to the standard output devices.

Syntax: `print(value(s), sep= ' ', end = '\n', file=file, flush=flush)`

Parameters:

value(s) : Any value, and as many as you like. Will be converted to string before printed
sep='separator' : (Optional) Specify how to separate the objects, if there is more than one. Default :
,

end='end' : (Optional) Specify what to print at the end. Default : '\n'

file : (Optional) An object with a write method. Default : `sys.stdout`

flush : (Optional) A Boolean, specifying if the output is flushed (True) or buffered (False). Default: False

Returns: It returns output to the screen.

Example: Python Print Output

- Python3

```
# Python program to demonstrate
```

```
# print() method
```

```
print("GFG")
```

```
# code for disabling the softspace feature
```

```
print('G', 'F', 'G')
```

Output

GFG

G F G

Example: Python Print output with custom sep and end parameter

- Python3

```
# Python program to demonstrate
```

```
# print() method
```

```
print("GFG", end = "@")
```

```
# code for disabling the softspace feature
```

```
print('G', 'F', 'G', sep="#")
```

Output

GFG@G#F#G

Formatting Output

Formatting output in Python can be done in many ways. Let's discuss them below

Using formatted string literals

We can use [formatted string literals](#), by starting a string with f or F before opening quotation marks or triple quotation marks. In this string, we can write Python expressions between { and } that can refer to a variable or any literal value.

Example: Python String formatting using F string

- Python3

```
# Declaring a variable
```

```
name = "Gfg"
```

```
# Output
```

```
print(f'Hello {name}! How are you?')
```

Output:

```
Hello Gfg! How are you?
```

Using format()

We can also use [format\(\)](#) function to format our output to make it look presentable. The curly braces { } work as placeholders. We can specify the order in which variables occur in the output.

Example: Python string formatting using format() function

- Python3

```
# Initializing variables
```

```
a = 20
```

```
b = 10
```

```
# addition
```

```
sum = a + b
```

```
# subtraction
```

```
sub = a - b
```

Output

```
print("The value of a is {} and b is {}".format(a,b))
```

```
print('{2} is the sum of {0} and {1}'.format(a,b,sum))
```

```
print('{sub_value} is the subtraction of {value_a} and {value_b}'.format(value_a =  
a ,value_b = b, sub_value = sub))
```

Output:

The value of a is 20 and b is 10

30 is the sum of 20 and 10

10 is the subtraction of 20 and 10

Using % Operator

We can use [‘%’ operator](#). % values are replaced with zero or more value of elements. The formatting using % is similar to that of ‘printf’ in the C programming language.

- %d – integer
- %f – float
- %s – string
- %x – hexadecimal
- %o – octal

Example:

- Python3

```
num = int(input("Enter a value: "))
```

```
add = num + 5
```

```
print("The sum is %d" %add)
```

Output:

Enter a value: 50

The sum is 55

Python Operators in general are used to perform operations on values and variables. These are standard symbols used for the purpose of logical and arithmetic operations. In this article, we will look into different types of Python operators.

- OPERATORS: Are the special symbols. Eg- + , * , /, etc.

- **OPERAND:** It is the value on which the operator is applied.

Arithmetic Operators

Arithmetic operators are used to performing mathematical operations like addition, subtraction, multiplication, and division.

- In Python 3.x the result of division is a floating-point while in Python 2.x division of 2 integer was an integer and to obtain an integer result in Python 3.x floored (`//` integer) is used.

Operator	Description	Syntax
+	Addition: adds two operands	<code>x + y</code>
-	Subtraction: subtracts two operands	<code>x - y</code>
*	Multiplication: multiplies two operands	<code>x * y</code>
/	Division (float): divides the first operand by the second	<code>x / y</code>
//	Division (floor): divides the first operand by the second	<code>x // y</code>
%	Modulus: returns the remainder when the first operand is divided by the second	<code>x % y</code>
**	Power: Returns first raised to power second	<code>x ** y</code>

PRECEDENCE:

- P – Parentheses
- E – Exponentiation
- M – Multiplication (Multiplication and division have the same precedence)
- D – Division
- A – Addition (Addition and subtraction have the same precedence)
- S – Subtraction

The modulus operator helps us extract the last digit/s of a number. For example:

- `x % 10` -> yields the last digit
- `x % 100` -> yield last two digits

Comparison Operators

Comparison of Relational operators compares the values. It either returns **True** or **False** according to the condition.

Operator	Description	Syntax
>	Greater than: True if the left operand is greater than the right	$x > y$
<	Less than: True if the left operand is less than the right	$x < y$
==	Equal to: True if both operands are equal	$x == y$
!=	Not equal to – True if operands are not equal	$x != y$
>=	Greater than or equal to True if the left operand is greater than or equal to the right	$x >= y$
<=	Less than or equal to True if the left operand is less than or equal to the right	$x <= y$
is	x is the same as y	x is y
is not	x is not the same as y	x is not y

= is an assignment operator and == comparison operator.

Logical Operators

Logical operators perform **Logical AND**, **Logical OR**, and **Logical NOT** operations. It is used to combine conditional statements.

Operator	Description	Syntax
and	Logical AND: True if both the operands are true	x and y
or	Logical OR: True if either of the operands is true	x or y
not	Logical NOT: True if the operand is false	not x

Bitwise Operators

Bitwise operators act on bits and perform the bit-by-bit operations. These are used to operate on binary numbers.

Operator	Description	Syntax
&	Bitwise AND	$x \& y$
	Bitwise OR	$x y$
~	Bitwise NOT	$\sim x$
^	Bitwise XOR	$x \wedge y$
>>	Bitwise right shift	$x >>$
<<	Bitwise left shift	$x <<$

Assignment Operators

Assignment operators are used to assign values to the variables.

Operator	Description	Syntax
=	Assign value of right side of expression to left side operand	$x = y + z$
+=	Add AND: Add right-side operand with left side operand and then assign to left operand	$a += b$ $a = a + b$
-=	Subtract AND: Subtract right operand from left operand and then assign to left operand	$a -= b$ $a = a - b$
*=	Multiply AND: Multiply right operand with left operand and then assign to left operand	$a *= b$ $a = a * b$
/=	Divide AND: Divide left operand with right operand and then assign to left operand	$a /= b$ $a = a / b$
%=	Modulus AND: Takes modulus using left and right operands and assign the result to left operand	$a \% = b$ $a = a \% b$
//=	Divide(floor) AND: Divide left operand with right operand and then assign the value(floor) to left operand	$a // = b$ $a = a // b$

Operator	Description	Syntax
<code>**=</code>	Exponent AND: Calculate exponent(raise power) value using operands and assign value to left operand	<code>a**=b</code> <code>a=a**b</code>
<code>&=</code>	Performs Bitwise AND on operands and assign value to left operand	<code>a&=b</code> <code>a=a&b</code>
<code> =</code>	Performs Bitwise OR on operands and assign value to left operand	<code>a =b</code> <code>a=a b</code>
<code>^=</code>	Performs Bitwise xOR on operands and assign value to left operand	<code>a^=b</code> <code>a=a^b</code>
<code>>>=</code>	Performs Bitwise right shift on operands and assign value to left operand	<code>a>>=b</code> <code>a=a>>b</code>
<code><<=</code>	Performs Bitwise left shift on operands and assign value to left operand	<code>a <<= b</code> <code>a= a << b</code>

Identity Operators

is and **is not** are the identity operators both are used to check if two values are located on the same part of the memory. Two variables that are equal do not imply that they are identical.

is True if the operands are identical

is not True if the operands are not identical

Membership Operators

in and **not in** are the membership operators; used to test whether a value or variable is in a sequence.

in True if value is found in the sequence

not in True if value is not found in the sequence

UNIT2: Control Flow Statements

Introduction to control structure

Control structures in Python are used to control the flow of execution in a program. They are essential for making decisions and repeating blocks of code. Here are the main types of control structures in Python:

1. **Sequential Flow:** This is the default mode of executing instructions one after the other in the order they appear in the script¹.
2. **Selection or Conditional Flow (if-elif-else):** This involves decision making and executing different parts of the code based on certain conditions¹. Here's a simple example:

```
v = 5
t = 4
if v > t:
    print(v, "is bigger than", t)
    v -= 2
```

3. **Iteration or Looping (for, while):** This involves executing a block of code repeatedly based on a condition¹. For example, a for loop in Python might look like this:

```
for i in range(5):
    print(i)
```

4. **Jump Statements (break, continue, pass):** These are used to skip parts of the loop or terminate it

Selection statements

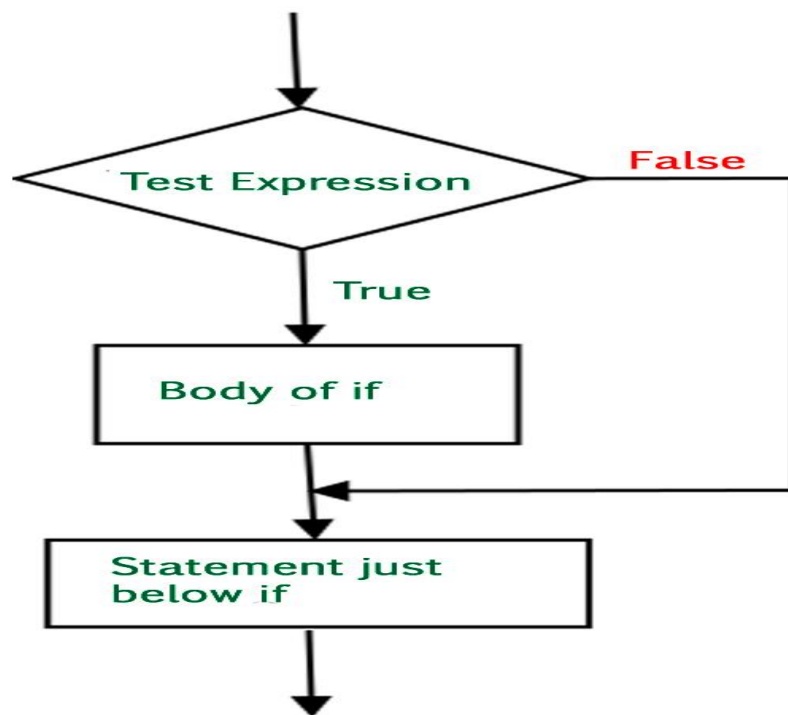
if Statement

If the simple code of block is to be performed if the condition holds true then the if statement is used. Here the condition mentioned holds true then the code of the block runs otherwise not.

Syntax:

```
if condition:
    # Statements to execute if
    # condition is true
```

Flowchart:-



Example:

```
# if statement example
if 10 > 5:
    print("10 greater than 5")

print("Program ended")
```

Output:

10 greater than 5

Program ended

Indentation (White space) is used to delimit the block of code. As shown in the above example it is mandatory to use indentation in Python3 coding.

if..else Statement

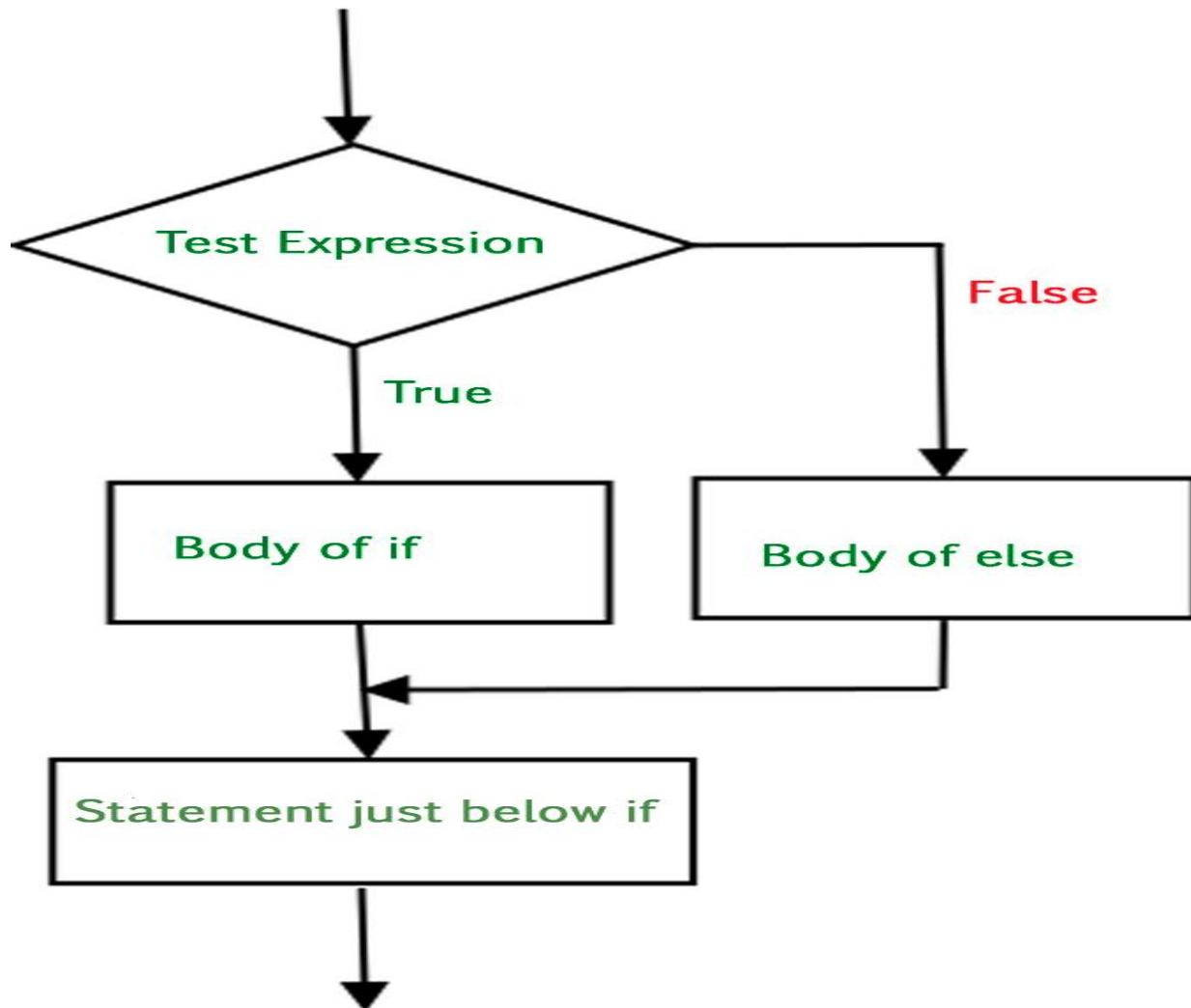
In conditional if Statement the additional block of code is merged as else statement which is performed when if condition is false.

Syntax:

```
if (condition):
    # Executes this block if
    # condition is true
else:
    # Executes this block if
```

condition is false

Flow Chart:-



Example 1:

```
# if..else statement example
x = 3
if x == 4:
    print("Yes")
else:
    print("No")
```

Output:

No

Example 2: You can also chain if..else statement with more than one condition.

```
# if..else chain statement
letter = "A"

if letter == "B":
    print("letter is B")

else:

    if letter == "C":
        print("letter is C")

    else:

        if letter == "A":
            print("letter is A")

        else:
            print("letter isn't A, B and C")
```

Output:

letter is A

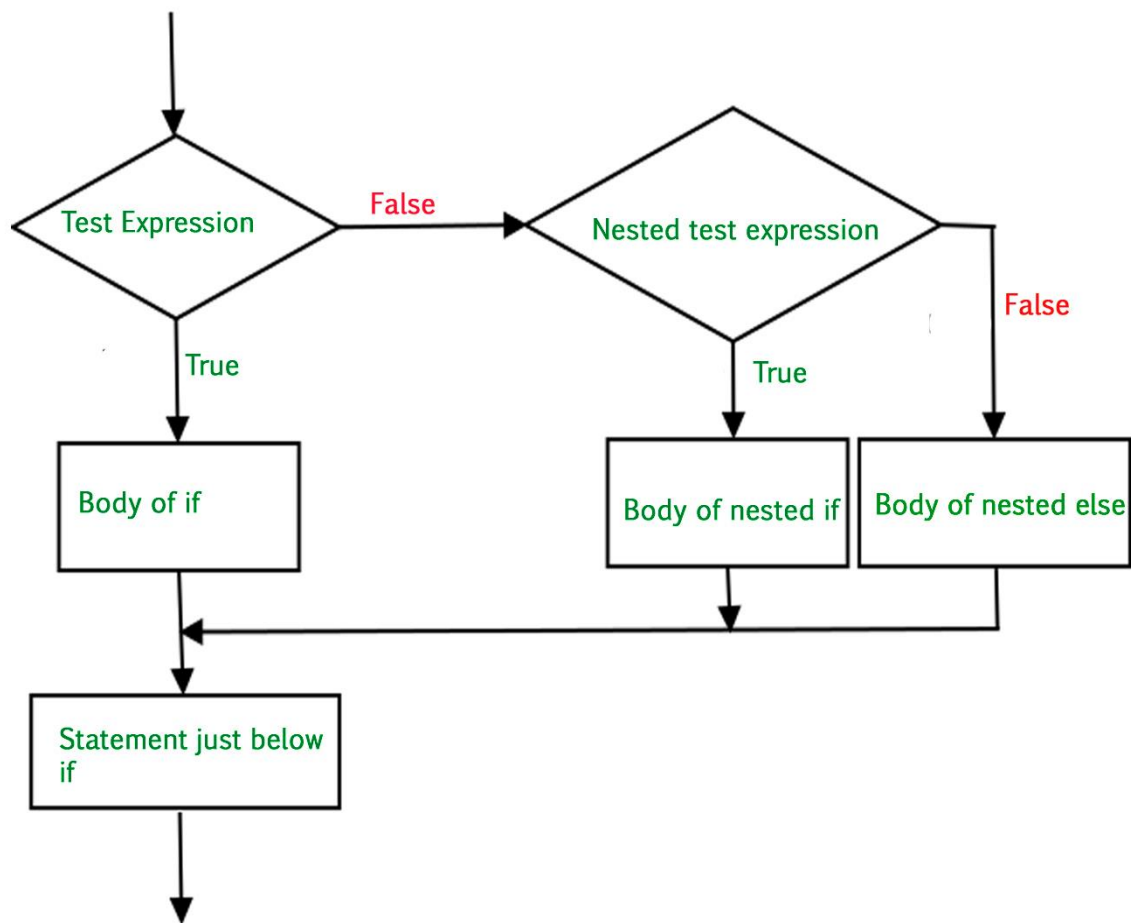
Nested if Statement

if statement can also be checked inside other if statement. This conditional statement is called a nested if statement. This means that inner if condition will be checked only if outer if condition is true and by this, we can see multiple conditions to be satisfied.

Syntax:

```
if (condition1):
    # Executes when condition1 is true
    if (condition2):
        # Executes when condition2 is true
    # if Block is end here
# if Block is end here
```

Flow chart:-



Example:

```
# Nested if statement example
num = 10

if num > 5:
    print("Bigger than 5")

    if num <= 15:
        print("Between 5 and 15")
```

Output:

Bigger than 5
Between 5 and 15

if-elif Statement

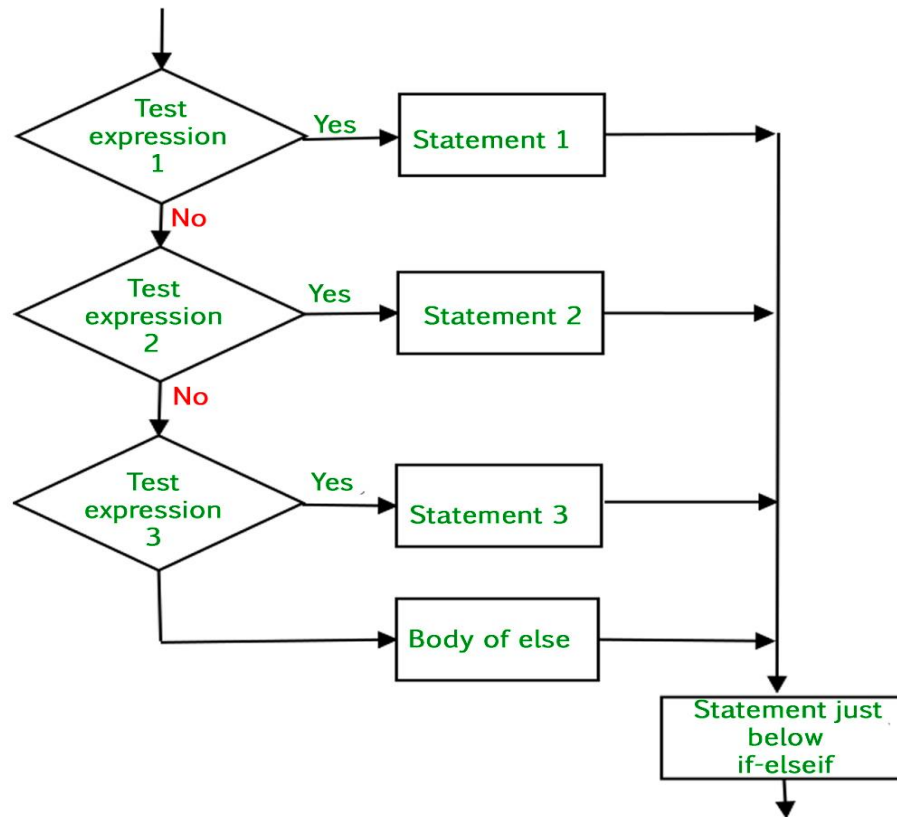
The if-elif statement is shortcut of if..else chain. While using if-elif statement at the end else block is added which is performed if none of the above if-elif statement is true.

Syntax:-

if (condition):

```
statement
elif (condition):
    statement
.
.
else:
    statement
```

Flow Chart:-



Example:-

```
# if-elif statement example

letter = "A"

if letter == "B":
    print("letter is B")

elif letter == "C":
    print("letter is C")

elif letter == "A":
    print("letter is A")
```

```
else:  
    print("letter isn't A, B or C")
```

Output:

letter is A

Loops in Python**While Loop in Python**

In python, a [while loop](#) is used to execute a block of statements repeatedly until a given condition is satisfied. And when the condition becomes false, the line immediately after the loop in the program is executed.

Syntax:

```
while expression:  
    statement(s)
```

All the statements indented by the same number of character spaces after a programming construct are considered to be part of a single block of code. Python uses indentation as its method of grouping statements.

Example of Python While Loop

```
# Python program to illustrate while loop  
count = 0  
while (count < 3):  
    count = count + 1  
    print("Hello Geek")
```

Output

Hello Geek
Hello Geek
Hello Geek

Using else statement with While Loop in Python

The else clause is only executed when your while condition becomes false. If you break out of the loop, or if an exception is raised, it won't be executed.

Syntax of While Loop with else statement:

```
while condition:  
    # execute these statements  
else:  
    # execute these statements
```

Examples of While Loop with else statement

Here is an example of while loop with else statement in Python:

```
# Python program to illustrate
```

```
# combining else with while
count = 0
while (count < 3):
    count = count + 1
    print("Hello Geek")
else:
    print("In Else Block")
```

Output

```
Hello Geek
Hello Geek
Hello Geek
In Else Block
```

Infinite While Loop in Python

If we want a block of code to execute infinite number of time, we can use the while loop in Python to do so.

```
# Python program to illustrate
# Single statement while block
count = 0
while (count == 0):
    print("Hello Geek")
```

Note: It is suggested **not to use** this type of loop as it is a never-ending infinite loop where the condition is always true and you have to forcefully terminate the compiler.

For Loop in Python

[For loops](#) are used for sequential traversal. For example: traversing a list or [string](#) or array etc. In Python, there is “for in” loop which is similar to [for each](#) loop in other languages. Let us learn how to use for in loop for sequential traversals.

Syntax:

```
for iterator_var in sequence:
    statements(s)
```

It can be used to iterate over a range and iterators.

```
# Python program to illustrate
# Iterating over range 0 to n-1

n = 4
for i in range(0, n):
    print(i)
```

Output

```
0
1
2
```

Example with List, Tuple, string, and dictionary iteration using For Loops in Python

```
# Python program to illustrate
# Iterating over a list
print("List Iteration")
l = ["geeks", "for", "geeks"]
for i in l:
    print(i)

# Iterating over a tuple (immutable)
print("\nTuple Iteration")
t = ("geeks", "for", "geeks")
for i in t:
    print(i)

# Iterating over a String
print("\nString Iteration")
s = "Geeks"
for i in s:
    print(i)

# Iterating over dictionary
print("\nDictionary Iteration")
d = dict()
d['xyz'] = 123
d['abc'] = 345
for i in d:
    print("%s %d" % (i, d[i]))
# Iterating over a set
print("\nSet Iteration")
set1 = {1, 2, 3, 4, 5, 6}
for i in set1:
    print(i),
```

Output

List Iteration

geeks
for
geeks

Tuple Iteration

geeks
for
geeks

String Iteration

G
e
e
k
s

Dictionary Iteration

xyz 123

abc 345

Set Iteration

1
2
3
4
5
6

Iterating by the index of sequences

We can also use the index of elements in the sequence to iterate. The key idea is to first calculate the length of the list and then iterate over the sequence within the range of this length. See the below example:

```
# Python program to illustrate
# Iterating by index
list = ["geeks", "for", "geeks"]
for index in range(len(list)):
    print(list[index])
```

Output

geeks
for
geeks

Using else statement with for loop in Python

We can also combine else statement with for loop like in while loop. But as there is no condition in for loop based on which the execution will terminate so the else block will be executed immediately after for block finishes execution.

```
# Python program to illustrate
# combining else with for

list = ["geeks", "for", "geeks"]
for index in range(len(list)):
    print(list[index])
else:
    print("Inside Else Block")
```

Output

```
geeks
for
geeks
Inside Else Block
Nested Loops
```

Python programming language allows to use one loop inside another loop. Following section shows few examples to illustrate the concept.

Syntax:

```
for iterator_var in sequence:
    for iterator_var in sequence:
        statements(s)
    statements(s)
```

The syntax for a nested while loop statement in the Python programming language is as follows:

```
while expression:
    while expression:
        statement(s)
    statement(s)
```

A final note on loop nesting is that we can put any type of loop inside of any other type of loop. For example, a for loop can be inside a while loop or vice versa.

```
# Python program to illustrate
# nested for loops in Python
from __future__ import print_function
for i in range(1, 5):
    for j in range(i):
        print(i, end=' ')
    print()
```

Output

```
1
2 2
3 3 3
4 4 4 4
```

Loop Control Statements

Loop control statements change execution from their normal sequence. When execution leaves a scope, all automatic objects that were created in that scope are destroyed. Python supports the following control statements.

Continue Statement

the [continue statement](#) in Python returns the control to the beginning of the loop.

```
# Prints all letters except 'e' and 's'
for letter in 'geeksforgeeks':
```

```
if letter == 'e' or letter == 's':  
    continue  
print('Current Letter :', letter)
```

Output

Current Letter : g
Current Letter : k
Current Letter : f
Current Letter : o
Current Letter : r
Current Letter : g
Current Letter : k

Break Statement

The [break statement](#) in Python brings control out of the loop.

```
for letter in 'geeksforgeeks':  
  
    # break the loop as soon it sees 'e'  
    # or 's'  
    if letter == 'e' or letter == 's':  
        break  
  
print('Current Letter :', letter)
```

Output

Current Letter : e

Pass Statement

We use [pass statement](#) in Python to write empty loops. Pass is also used for empty control statements, functions and classes.

```
# An empty loop  
for letter in 'geeksforgeeks':  
    pass  
print('Last Letter :', letter)
```

Output

Last Letter : s

How for loop in Python works internally?

Before proceeding to this section, you should have a prior understanding of Python Iterators. Firstly, lets see how a simple for loop looks like.

```
# A simple for loop example
```

```
fruits = ["apple", "orange", "kiwi"]
```

```
for fruit in fruits:
```

```
    print(fruit)
```

Output

```
apple
orange
kiwi
```

Here we can see the for loops iterates over iterable object fruit which is a list. Lists, sets, dictionaries are few iterable objects while an integer object is not an iterable object. For loops can iterate over any of these iterable objects.

Now with the help of the above example, let's dive deep and see what happens internally here.

1. Make the list (iterable) an iterable object with help of the iter() function.
2. Run an infinite while loop and break only if the StopIteration is raised.
3. In the try block, we fetch the next element of fruits with the next() function.
4. After fetching the element we did the operation to be performed with the element. (i.e print(fruit))

```
fruits = ["apple", "orange", "kiwi"]
```

```
# Creating an iterator object
```

```
# from that iterable i.e fruits
```

```
iter_obj = iter(fruits)
```

```
# Infinite while loop
```

```
while True:
```

```
    try:
```

```
        # getting the next item
```

```
        fruit = next(iter_obj)
```

```
        print(fruit)
```

```
    except StopIteration:
```

```
        # if StopIteration is raised,
```

```
        # break from loop
```

```
        break
```

Output

```
apple
orange
kiwi
```

UNIT – 3: Lists, Tuple, Sets and Dictionaries

Python Lists

List is a built-in data structure used to store an ordered collection of items. They are dynamic, resizable and capable of storing multiple data types.

- Mutable: list elements can be changed, updated, added, or removed after the list is created.
- Ordered: elements maintain the order in which they are inserted.
- Index-based: elements are accessed using their position, starting from index 0.

In simple language, a list is a collection of things, enclosed in [] and separated by commas.

The list is a sequence data type which is used to store the collection of data. [Tuples](#) and [String](#) are other types of sequence data types.

Example of list in Python

```
Var = ["Geeks", "for", "Geeks"]  
print(Var)
```

Output:

```
["Geeks", "for", "Geeks"]
```

Lists are the simplest containers that are an integral part of the Python language. Lists need not be homogeneous always which makes it the most powerful tool in [Python](#). A single list may contain DataTypes like Integers, Strings, as well as Objects. Lists are mutable, and hence, they can be altered even after their creation.

Creating a List in Python

Lists in Python can be created by just placing the sequence inside the square brackets[]. Unlike [Sets](#), a list doesn't need a built-in function for its creation of a list.

Note: *Unlike Sets, the list may contain mutable elements.*

Example 1: Creating a list in Python

```
# Python program to demonstrate  
List = []  
print("Blank List: ")  
print(List)  
# Creating a List of numbers  
List = [10, 20, 14]  
print("\nList of numbers: ")  
print(List)  
# Creating a List of strings and accessing  
# using index  
List = ["Geeks", "For", "Geeks"]  
print("\nList Items: ")  
print(List[0])  
print(List[2])
```

Output

Blank List:

```
[]
```

List of numbers:

```
[10, 20, 14]
```

List Items:

```
Geeks
```

```
Geeks
```

Example 2: Creating a list with multiple distinct or duplicate elements

A list may contain duplicate values with their distinct positions and hence, multiple distinct or duplicate values can be passed as a sequence at the time of list creation.

```
# Creating a List with
# the use of Numbers
# (Having duplicate values)
List = [1, 2, 4, 4, 3, 3, 3, 6, 5]
print("\nList with the use of Numbers: ")
print(List)

# Creating a List with
# mixed type of values
# (Having numbers and strings)
List = [1, 2, 'Geeks', 4, 'For', 6, 'Geeks']
print("\nList with the use of Mixed Values: ")
print(List)
```

Output

List with the use of Numbers:

```
[1, 2, 4, 4, 3, 3, 3, 6, 5]
```

List with the use of Mixed Values:

```
[1, 2, 'Geeks', 4, 'For', 6, 'Geeks']
```

Accessing elements from the List

In order to access the list items refer to the index number. Use the index operator [] to access an item in a list. The index must be an integer. Nested lists are accessed using nested indexing.

Example 1: Accessing elements from list

```
# Python program to demonstrate
# accessing of element from list

# Creating a List with
# the use of multiple values
List = ["Geeks", "For", "Geeks"]

# accessing a element from the
```

```
# list using index number
print("Accessing a element from the list")
print(List[0])
print(List[2])
```

Output

Accessing a element from the list
Geeks
Geeks

Example 2: Accessing elements from a multi-dimensional list

```
# Creating a Multi-Dimensional List
# (By Nesting a list inside a List)
List = [['Geeks', 'For'], ['Geeks']]

# accessing an element from the
# Multi-Dimensional List using
# index number
print("Accessing a element from a Multi-Dimensional list")
print(List[0][1])
print(List[1][0])
```

Output

Accessing a element from a Multi-Dimensional list
For
Geeks

Negative indexing

In Python, negative sequence indexes represent positions from the end of the array. Instead of having to compute the offset as in `List[len(List)-3]`, it is enough to just write `List[-3]`. Negative indexing means beginning from the end, -1 refers to the last item, -2 refers to the second-last item, etc.

```
List = [1, 2, 'Geeks', 4, 'For', 6, 'Geeks']
print("Accessing element using negative indexing")
print(List[-1])
print(List[-3])
```

Output

Accessing element using negative indexing
Geeks
For

Getting the size of Python list

Python [len\(\)](#) is used to get the length of the list.

```
# Creating a List
List1 = []
print(len(List1))

# Creating a List of numbers
List2 = [10, 20, 14]
print(len(List2))
```

Output

```
0
3
```

Taking Input of a Python List

We can take the input of a list of elements as string, integer, float, etc. But the default one is a string.

Example 1:

```
# Python program to take space
# separated input as a string
# split and store it to a list
# and print the string list

# input the list as string
string = input("Enter elements (Space-Separated): ")

# split the strings and store it to a list
lst = string.split()
print("The list is:", lst) # printing the list
```

Output:

```
Enter elements: GEEKS FOR GEEKS
The list is: ['GEEKS', 'FOR', 'GEEKS']
```

Example 2:

```
# input size of the list
n = int(input("Enter the size of list : "))
# store integers in a list using map,
# split and strip functions
lst = list(map(int, input("Enter the integer\
elements:").strip().split()))[:n]
```

```
# printing the list
print("The list is:", lst)
```

Output:

Enter the size of list : 4

Enter the integer elements: 6 3 9 10

The list is: [6, 3, 9, 10]

Adding Elements to a Python List

Method 1: Using append() method

Elements can be added to the List by using the built-in **append()** function. Only one element at a time can be added to the list by using the append() method, for the addition of multiple elements with the append() method, loops are used. Tuples can also be added to the list with the use of the append method because tuples are immutable. Unlike Sets, Lists can also be added to the existing list with the use of the append() method.

```
# Python program to demonstrate
# Addition of elements in a List

# Creating a List
List = []
print("Initial blank List: ")
print(List)

# Addition of Elements
# in the List
List.append(1)
List.append(2)
List.append(4)
print("\nList after Addition of Three elements: ")
print(List)

# Adding elements to the List
# using Iterator
for i in range(1, 4):
    List.append(i)
print("\nList after Addition of elements from 1-3: ")
print(List)

# Adding Tuples to the List
List.append((5, 6))
```

```
print("\nList after Addition of a Tuple: ")
print(List)

# Addition of List to a List
List2 = ['For', 'Geeks']
List.append(List2)
print("\nList after Addition of a List: ")
print(List)
```

Output

Initial blank List:

```
[]
```

List after Addition of Three elements:

```
[1, 2, 4]
```

List after Addition of elements from 1-3:

```
[1, 2, 4, 1, 2, 3]
```

List after Addition of a Tuple:

```
[1, 2, 4, 1, 2, 3, (5, 6)]
```

List after Addition of a List:

```
[1, 2, 4, 1, 2, 3, (5, 6), ['For', 'Geeks']]
```

Method 2: Using insert() method

append() method only works for the addition of elements at the end of the List, for the addition of elements at the desired position, [insert\(\)](#) method is used. Unlike append() which takes only one argument, the insert() method requires two arguments(position, value).

```
# Python program to demonstrate
# Addition of elements in a List

# Creating a List
List = [1,2,3,4]
print("Initial List: ")
print(List)
```

```
# Addition of Element at
# specific Position
# (using Insert Method)
List.insert(3, 12)
List.insert(0, 'Geeks')
print("\nList after performing Insert Operation: ")
print(List)
```

Output

Initial List:

[1, 2, 3, 4]

List after performing Insert Operation:

['Geeks', 1, 2, 3, 12, 4]

Method 3: Using extend() method

Other than append() and insert() methods, there's one more method for the Addition of elements, **extend()**, this method is used to add multiple elements at the same time at the end of the list.

Note: [append\(\)](#) and [extend\(\)](#) methods can only add elements at the end.

```
# Python program to demonstrate
# Addition of elements in a List

# Creating a List
List = [1, 2, 3, 4]
print("Initial List: ")
print(List)

# Addition of multiple elements
# to the List at the end
# (using Extend Method)
List.extend([8, 'Geeks', 'Always'])
print("\nList after performing Extend Operation: ")
print(List)
```

Output

Initial List:

[1, 2, 3, 4]

List after performing Extend Operation:

[1, 2, 3, 4, 8, 'Geeks', 'Always']

Reversing a List

Method 1: A list can be reversed by using the [reverse\(\) method in Python](#).

```
# Reversing a list
mylist = [1, 2, 3, 4, 5, 'Geek', 'Python']
mylist.reverse()
print(mylist)
```

Output

```
['Python', 'Geek', 5, 4, 3, 2, 1]
```

Method 2: Using the [reversed\(\) function](#):

The reversed() function returns a reverse iterator, which can be converted to a list using the list() function.

```
my_list = [1, 2, 3, 4, 5]
reversed_list = list(reversed(my_list))
print(reversed_list)
```

Output

```
[5, 4, 3, 2, 1]
```

Removing Elements from the List

Method 1: Using remove() method

Elements can be removed from the List by using the built-in **remove()** function but an Error arises if the element doesn't exist in the list. Remove() method only removes one element at a time, to remove a range of elements, the iterator is used. The remove() method removes the specified item.

Note: Remove method in List will only remove the first occurrence of the searched element.

Example 1:

```
# Python program to demonstrate
# Removal of elements in a List

# Creating a List
List = [1, 2, 3, 4, 5, 6,
        7, 8, 9, 10, 11, 12]
print("Initial List: ")
print(List)

# Removing elements from List
```

```
# using Remove() method
List.remove(5)
List.remove(6)
print("\nList after Removal of two elements: ")
print(List)
```

Output

Initial List:

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]

List after Removal of two elements:

[1, 2, 3, 4, 7, 8, 9, 10, 11, 12]

Example 2:

```
# Creating a List
List = [1, 2, 3, 4, 5, 6,
        7, 8, 9, 10, 11, 12]
# Removing elements from List
# using iterator method
for i in range(1, 5):
    List.remove(i)
print("\nList after Removing a range of elements: ")
print(List)
```

Output

List after Removing a range of elements:

[5, 6, 7, 8, 9, 10, 11, 12]

Method 2: Using pop() method

[pop\(\) function](#) can also be used to remove and return an element from the list, but by default it removes only the last element of the list, to remove an element from a specific position of the List, the index of the element is passed as an argument to the pop() method.

```
List = [1, 2, 3, 4, 5]

# Removing element from the
# Set using the pop() method
List.pop()
print("\nList after popping an element: ")
print(List)

# Removing element at a
# specific location from the
# Set using the pop() method
```

```
List.pop(2)
print("\nList after popping a specific element: ")
print(List)
```

Output

List after popping an element:
[1, 2, 3, 4]

List after popping a specific element:
[1, 2, 4]

Slicing of a List

We can get substrings and sublists using a slice. In Python List, there are multiple ways to print the whole list with all the elements, but to print a specific range of elements from the list, we use the [Slice operation](#).

Slice operation is performed on Lists with the use of a colon(:).

To print elements from beginning to a range use:

`[ : Index]`

To print elements from end-use:

`[ :-Index]`

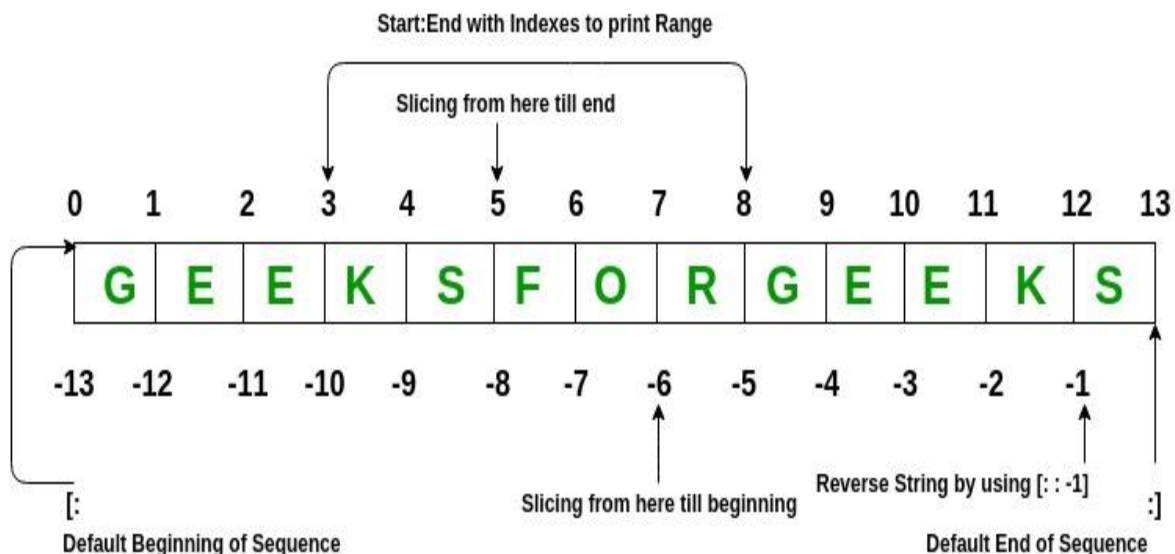
To print elements from a specific Index till the end use

`[Index: ]`

To print the whole list in reverse order, use

`[::-1]`

Note – To print elements of List from rear-end, use Negative Indexes.



UNDERSTANDING SLICING OF LISTS:

- `pr[0]` accesses the first item, 2.
- `pr[-4]` accesses the fourth item from the end, 5.

- pr[2:] accesses [5, 7, 11, 13], a list of items from third to last.
- pr[:4] accesses [2, 3, 5, 7], a list of items from first to fourth.
- pr[2:4] accesses [5, 7], a list of items from third to fifth.
- pr[1::2] accesses [3, 7, 13], alternate items, starting from the second item.

```
# Python program to demonstrate
# Removal of elements in a List

# Creating a List
List = ['G', 'E', 'E', 'K', 'S', 'F',
        'O', 'R', 'G', 'E', 'E', 'K', 'S']
print("Initial List: ")
print(List)

# Print elements of a range
# using Slice operation
Sliced_List = List[3:8]
print("\nSlicing elements in a range 3-8: ")
print(Sliced_List)

# Print elements from a
# pre-defined point to end
Sliced_List = List[5:]
print("\nElements sliced from 5th "
      "element till the end: ")
print(Sliced_List)

# Printing elements from
# beginning till end
Sliced_List = List[:]
print("\nPrinting all elements using slice operation: ")
print(Sliced_List)
```

Output

```
Initial List:
['G', 'E', 'E', 'K', 'S', 'F', 'O', 'R', 'G', 'E', 'E', 'K', 'S']
Slicing elements in a range 3-8:
['K', 'S', 'F', 'O', 'R']
Elements sliced from 5th element till the end:
['F', 'O', 'R', 'G', 'E', 'E', 'K', 'S']
Printing all elements using slice operation:
['G', 'E', 'E', 'K', 'S', 'F', 'O', 'R', 'G', 'E', 'E', 'K', 'S']
```

Negative index List slicing

```
# Creating a List
List = ['G', 'E', 'E', 'K', 'S', 'F',
        'O', 'R', 'G', 'E', 'E', 'K', 'S']
print("Initial List: ")
print(List)

# Print elements from beginning
# to a pre-defined point using Slice
Sliced_List = List[:-6]
print("\nElements sliced till 6th element from last: ")
print(Sliced_List)

# Print elements of a range
# using negative index List slicing
Sliced_List = List[-6:-1]
print("\nElements sliced from index -6 to -1")
print(Sliced_List)

# Printing elements in reverse
# using Slice operation
Sliced_List = List[::-1]
print("\nPrinting List in reverse: ")
print(Sliced_List)
```

Output

Initial List:
['G', 'E', 'E', 'K', 'S', 'F', 'O', 'R', 'G', 'E', 'E', 'K', 'S']
Elements sliced till 6th element from last:
['G', 'E', 'E', 'K', 'S', 'F', 'O']
Elements sliced from index -6 to -1
['R', 'G', 'E', 'E', 'K']
Printing List in reverse:
['S', 'K', 'E', 'E', 'G', 'R', 'O', 'F', 'S', 'K', 'E', 'E', 'G']

List Methods

Function	Description
Append()	Add an element to the end of the list
Extend()	Add all elements of a list to another list

Function	Description
<u>Insert()</u>	Insert an item at the defined index
<u>Remove()</u>	Removes an item from the list
<u>Clear()</u>	Removes all items from the list
<u>Index()</u>	Returns the index of the first matched item
<u>Count()</u>	Returns the count of the number of items passed as an argument
<u>Sort()</u>	Sort items in a list in ascending order
<u>Reverse()</u>	Reverse the order of items in the list
<u>copy()</u>	Returns a copy of the list
<u>pop()</u>	Removes and returns the item at the specified index. If no index is provided, it removes and returns the last item.

Built-in functions with List

Function	Description
<u>reduce()</u>	apply a particular function passed in its argument to all of the list elements stores the intermediate result and only returns the final summation value

Function	Description
<u>sum()</u>	Sums up the numbers in the list
<u>ord()</u>	Returns an integer representing the Unicode code point of the given Unicode character
<u>cmp()</u>	This function returns 1 if the first list is “greater” than the second list
<u>max()</u>	return maximum element of a given list
<u>min()</u>	return minimum element of a given list
<u>all()</u>	Returns true if all element is true or if the list is empty
<u>any()</u>	return true if any element of the list is true. if the list is empty, return false
<u>len()</u>	Returns length of the list or size of the list
<u>enumerate()</u>	Returns enumerate object of the list
<u>accumulate()</u>	apply a particular function passed in its argument to all of the list elements returns a list containing the intermediate results
<u>filter()</u>	tests if each element of a list is true or not

Function	Description
map()	returns a list of the results after applying the given function to each item of a given iterable
lambda()	This function can have any number of arguments but only one expression, which is evaluated and returned.

Sets in Python

A Set in [Python programming](#) is an unordered collection data type that is iterable, mutable and has no duplicate elements.

Set are represented by { } (values enclosed in curly braces)

The major advantage of using a set, as opposed to a [list](#), is that it has a highly optimized method for checking whether a specific element is contained in the set. This is based on a data structure known as a [hash table](#). Since sets are unordered, we cannot access items using indexes as we do in lists.

Example of Python Sets

```
var = {"Geeks", "for", "Geeks"}
type(var)
```

Output:

set

Methods for Sets

Adding elements to Python Sets

Insertion in the set is done through the [set.add\(\)](#) function, where an appropriate record value is created to store in the hash table. Same as checking for an item, i.e., $O(1)$ on average. However, in worst case it can become $O(n)$.

```
# A Python program to
# demonstrate adding elements
# in a set

# Creating a Set
people = {"Jay", "Idrish", "Archi"}
```

```
print("People:", end = " ")
print(people)

# This will add Daxit
# in the set
people.add("Daxit")

# Adding elements to the
# set using iterator
for i in range(1, 6):
    people.add(i)

print("\nSet after adding element:", end = " ")
print(people)
```

Output:

People: {'Idrish', 'Archi', 'Jay'}

Set after adding element: {1, 2, 3, 4, 5, 'Idrish', 'Archi', 'Jay', 'Daxit'}

Union operation on Python Sets

Two sets can be merged using union() function or | operator. Both Hash Table values are accessed and traversed with merge operation perform on them to combine the elements, at the same time duplicates are removed. The Time Complexity of this is **$O(\text{len}(s1) + \text{len}(s2))$** where s1 and s2 are two sets whose union needs to be done.

```
# Python Program to
# demonstrate union of
# two sets

people = {"Jay", "Idrish", "Archil"}
vampires = {"Karan", "Arjun"}
dracula = {"Deepanshu", "Raju"}

# Union using union()
# function
population = people.union(vampires)

print("Union using union() function")
print(population)

# Union using "|"
# operator
```

```
population = people|dracula

print("\nUnion using '|' operator")
print(population)
```

Output:

Union using union() function
{'Karan', 'Idrish', 'Jay', 'Arjun', 'Archil'}

Union using '|' operator
{'Deepanshu', 'Idrish', 'Jay', 'Raju', 'Archil'}

Intersection operation on Python Sets

This can be done through intersection() or & operator. Common Elements are selected. They are similar to iteration over the Hash lists and combining the same values on both the Table. Time Complexity of this is $O(\min(\text{len}(s1), \text{len}(s2)))$ where $s1$ and $s2$ are two sets whose union needs to be done.

```
# Python program to
# demonstrate intersection
# of two sets

set1 = set()
set2 = set()

for i in range(5):
    set1.add(i)

for i in range(3,9):
    set2.add(i)

# Intersection using
# intersection() function
set3 = set1.intersection(set2)

print("Intersection using intersection() function")
print(set3)

# Intersection using
# "&" operator
set3 = set1 & set2

print("\nIntersection using '&' operator")
print(set3)
```

Output:

Intersection using intersection() function

{3, 4}

Intersection using '&' operator

{3, 4}

Time

Complexity:

O(n)

Auxiliary Space: O(n)

Finding Differences of Sets in Python

To find differences between sets. Similar to finding differences in the linked list. This is done through difference() or – operator. Time complexity of finding difference $s1 - s2$ is $O(\text{len}(s1))$

```
# Python program to
# demonstrate difference
# of two sets

set1 = set()
set2 = set()

for i in range(5):
    set1.add(i)

for i in range(3,9):
    set2.add(i)

# Difference of two sets
# using difference() function
set3 = set1.difference(set2)

print(" Difference of two sets using difference() function")
print(set3)

# Difference of two sets
# using '-' operator
set3 = set1 - set2

print("\nDifference of two sets using '-' operator")
print(set3)
```

Output:

Difference of two sets using difference() function

{0, 1, 2}

Difference of two sets using '-' operator

{0, 1, 2}

Time
Auxiliary Space: O(n)

Complexity:

O(n)

Clearing Python Sets

Set Clear() method empties the whole set inplace.

```
# Python program to
# demonstrate clearing
# of set

set1 = {1,2,3,4,5,6}

print("Initial set")
print(set1)

# This method will remove
# all the elements of the set
set1.clear()

print("\nSet after using clear() function")
print(set1)
```

Output:

Initial set
{1, 2, 3, 4, 5, 6}

Set after using clear() function
set()

Time
Auxiliary Space: O(n)

Complexity:

O(n)

Tuples in Python

Python Tuple is a collection of objects separated by commas. In some ways, a tuple is similar to a Python list in terms of indexing, nested objects, and repetition but the main difference between both is Python tuple is immutable, unlike the Python list which is mutable.

Creating Python Tuples

There are various ways by which you can create a tuple in [Python](#). They are as follows:

- Using round brackets
- With one item
- Tuple Constructor

Create Tuples using Round Brackets ()

To create a tuple we will use () operators.

```
var = ("Geeks", "for", "Geeks")
print(var)
```

Output:

('Geeks', 'for', 'Geeks')

Output:

<class 'tuple'>

<class 'str'>

Tuples in Python provide two ways by which we can access the elements of a tuple.

- Using a positive index
- Using a negative index

Python Access Tuple using a Positive Index

Using square brackets we can get the values from tuples in Python.

```
var = ("Geeks", "for", "Geeks")

print("Value in Var[0] = ", var[0])
print("Value in Var[1] = ", var[1])
print("Value in Var[2] = ", var[2])
```

Output:

Value in Var[0] = Geeks

Value in Var[1] = for

Value in Var[2] = Geeks

Access Tuple using Negative Index

In the above methods, we use the positive index to access the value in Python, and here we will use the negative index within [].

```
var = (1, 2, 3)

print("Value in Var[-1] = ", var[-1])
print("Value in Var[-2] = ", var[-2])
print("Value in Var[-3] = ", var[-3])
```

Output:

Value in Var[-1] = 3

Value in Var[-2] = 2

Value in Var[-3] = 1

Different Operations Related to Tuples

Below are the different operations related to tuples in Python:

- Concatenation
- Nesting
- Repetition
- Slicing
- Deleting
- Finding the length
- Multiple Data Types with tuples
- Conversion of lists to tuples
- Tuples in a Loop

Concatenation of Python Tuples

To [Concatenation](#) of Python Tuples, we will use plus operators(+).

```
# Code for concatenating 2 tuples
tuple1 = (0, 1, 2, 3)
tuple2 = ('python', 'geek')

# Concatenating above two
print(tuple1 + tuple2)
```

Output:

(0, 1, 2, 3, 'python', 'geek')

Nesting of Python Tuples

A nested tuple in Python means a tuple inside another tuple.

```
# Code for creating nested tuples
tuple1 = (0, 1, 2, 3)
tuple2 = ('python', 'geek')

tuple3 = (tuple1, tuple2)
print(tuple3)
```

Output :

((0, 1, 2, 3), ('python', 'geek'))

Repetition Python Tuples

We can create a tuple of multiple same elements from a single element in that tuple.

```
# Code to create a tuple with repetition
tuple3 = ('python',)*3
print(tuple3)
```

Output:

('python', 'python', 'python')

Try the above without a comma and check. You will get tuple3 as a string 'pythonpythonpython'.

Slicing Tuples in Python

Slicing a Python tuple means dividing a tuple into small tuples using the indexing method.

```
# code to test slicing
tuple1 = (0, 1, 2, 3)

print(tuple1[1:])
print(tuple1[::-1])
print(tuple1[2:4])
```

Output:

In this example, we sliced the tuple from index 1 to the last element. In the second print statement, we printed the tuple using reverse indexing. And in the third print statement, we printed the elements from index 2 to 4.

(1, 2, 3)

(3, 2, 1, 0)

(2, 3)

Note: In Python slicing, the end index provided is not included.

Deleting a Tuple in Python

In this example, we are deleting a tuple using '[del](#)' keyword. The output will be in the form of error because after deleting the tuple, it will give a NameError.

Note: Remove individual tuple elements is not possible, but we can delete the whole Tuple using Del keyword.

```
# Code for deleting a tuple
tuple3 = (0, 1)

del tuple3
print(tuple3)
```

Output:

Traceback (most recent call last):

```
File "d92694727db1dc9118a5250bf04dafbd.py", line 6, in <module>
    print(tuple3)
NameError: name 'tuple3' is not defined
```

Finding the Length of a Python Tuple

To find the length of a tuple, we can use Python's `len()` function and pass the tuple as the parameter.

```
# Code for printing the length of a tuple
tuple2 = ('python', 'geek')
print(len(tuple2))
```

Output:

```
2
```

Multiple Data Types With Tuple

Tuples in Python are heterogeneous in nature. This means tuples support elements with multiple datatypes.

```
# tuple with different datatypes
tuple_obj = ("immutable", True, 23)
print(tuple_obj)
```

Output :

```
('immutable', True, 23)
```

Converting a List to a Tuple

We can convert a [list](#) in Python to a tuple by using the `tuple()` constructor and passing the list as its parameters.

```
# Code for converting a list and a string into a tuple
list1 = [0, 1, 2]

print(tuple(list1))

# string 'python'
print(tuple('python'))
```

Output:

Tuples take a single parameter which may be a list, string, set, or even a dictionary (only keys are taken as elements), and converts them to a tuple.

```
(0, 1, 2)
```

```
('p', 'y', 't', 'h', 'o', 'n')
```

Python Dictionary

Dictionary in Python is a collection of keys values, used to store data values like a map, which, unlike other data types which hold only a single value as an element.

Example of Dictionary in Python

Dictionary holds **key:value** pair. Key-Value is provided in the dictionary to make it more optimized.

```
Dict = {1: 'Geeks', 2: 'For', 3: 'Geeks'}  
  
print(Dict)
```

Output:

```
{1: 'Geeks', 2: 'For', 3: 'Geeks'}
```

Creating a Dictionary

In [Python](#), a dictionary can be created by placing a sequence of elements within curly **{}** braces, separated by 'comma'. Dictionary holds pairs of values, one being the Key and the other corresponding pair element being its **Key:value**. Values in a dictionary can be of any data type and can be duplicated, whereas keys can't be repeated and must be *immutable*.

Note – Dictionary keys are case sensitive, the same name but different cases of Key will be treated distinctly.

```
# Creating a Dictionary  
# with Integer Keys  
Dict = {1: 'Geeks', 2: 'For', 3: 'Geeks'}  
print("\nDictionary with the use of Integer Keys: ")  
print(Dict)  
  
# Creating a Dictionary  
# with Mixed keys  
Dict = {'Name': 'Geeks', 1: [1, 2, 3, 4]}  
print("\nDictionary with the use of Mixed Keys: ")  
print(Dict)
```

Output:

Dictionary with the use of Integer Keys:

```
{1: 'Geeks', 2: 'For', 3: 'Geeks'}
```

Dictionary with the use of Mixed Keys:

```
{'Name': 'Geeks', 1: [1, 2, 3, 4]}
```

Dictionary can also be created by the built-in function `dict()`. An empty dictionary can be created by just placing curly braces `{}`.

```
# Creating an empty Dictionary
Dict = {}
print("Empty Dictionary: ")
print(Dict)

# Creating a Dictionary
# with dict() method
Dict = dict({1: 'Geeks', 2: 'For', 3: 'Geeks'})
print("\nDictionary with the use of dict(): ")
print(Dict)

# Creating a Dictionary
# with each item as a Pair
Dict = dict([(1, 'Geeks'), (2, 'For')])
print("\nDictionary with each item as a pair: ")
print(Dict)
```

Output:

Empty Dictionary:

```
{}
```

Dictionary with the use of dict():

```
{1: 'Geeks', 2: 'For', 3: 'Geeks'}
```

Dictionary with each item as a pair:

```
{1: 'Geeks', 2: 'For'}
```

Adding elements to a Dictionary

Addition of elements can be done in multiple ways. One value at a time can be added to a Dictionary by defining value along with the key e.g. Dict[Key] = 'Value'. Updating an existing value in a Dictionary can be done by using the built-in **update()** method. Nested key values can also be added to an existing Dictionary.

Note- While adding a value, if the key-value already exists, the value gets updated otherwise a new Key with the value is added to the Dictionary.

```
# Creating an empty Dictionary
Dict = {}
print("Empty Dictionary: ")
print(Dict)

# Adding elements one at a time
Dict[0] = 'Geeks'
Dict[2] = 'For'
Dict[3] = 1
```

```

print("\nDictionary after adding 3 elements: ")
print(Dict)

# Adding set of values
# to a single Key
Dict['Value_set'] = 2, 3, 4
print("\nDictionary after adding 3 elements: ")
print(Dict)

# Updating existing Key's Value
Dict[2] = 'Welcome'
print("\nUpdated key value: ")
print(Dict)

# Adding Nested Key value to Dictionary
Dict[5] = {'Nested': {'1': 'Life', '2': 'Geeks'}}
print("\nAdding a Nested Key: ")
print(Dict)

```

Output:

```

Empty Dictionary:
{}
Dictionary after adding 3 elements:
{0: 'Geeks', 2: 'For', 3: 1}
Dictionary after adding 3 elements:
{0: 'Geeks', 2: 'For', 3: 1, 'Value_set': (2, 3, 4)}
Updated key value:
{0: 'Geeks', 2: 'Welcome', 3: 1, 'Value_set': (2, 3, 4)}
Adding a Nested Key:
{0: 'Geeks', 2: 'Welcome', 3: 1, 'Value_set': (2, 3, 4), 5:
{'Nested': {'1': 'Life', '2': 'Geeks'}}}

```

Accessing elements of a Dictionary

In order to access the items of a dictionary refer to its key name. Key can be used inside square brackets.

```

# Python program to demonstrate
# accessing a element from a Dictionary

# Creating a Dictionary
Dict = {1: 'Geeks', 'name': 'For', 3: 'Geeks'}

# accessing a element using key

```

```
print("Accessing a element using key:")
print(Dict['name'])

# accessing a element using key
print("Accessing a element using key:")
print(Dict[1])
```

Output:

Accessing a element using key:

For

Accessing a element using key:

Geeks

There is also a method called **get()** that will also help in accessing the element from a dictionary. This method accepts key as argument and returns the value.

```
# Creating a Dictionary
Dict = {1: 'Geeks', 'name': 'For', 3: 'Geeks'}

# accessing a element using get()
# method
print("Accessing a element using get:")
print(Dict.get(3))
```

Output:

Accessing a element using get:

Geeks

Deleting Elements using del Keyword

The items of the dictionary can be deleted by using the del keyword as given below.

```
# Python program to demonstrate
# Deleting Elements using del Keyword

# Creating a Dictionary
Dict = {1: 'Geeks', 'name': 'For', 3: 'Geeks'}

print("Dictionary =")
print(Dict)
#Deleting some of the Dictionar data
del(Dict[1])
print("Data after deletion Dictionary=")
```

```
print(Dict)
```

Output

Dictionary ={1: 'Geeks', 'name': 'For', 3: 'Geeks'}

Data after deletion Dictionary={'name': 'For', 3: 'Geeks'}

Dictionary methods

Method	Description
dic.clear()	Remove all the elements from the dictionary
dict.copy()	Returns a copy of the dictionary
dict.get(key, default = "None")	Returns the value of specified key
dict.items()	Returns a list containing a tuple for each key value pair
dict.keys()	Returns a list containing dictionary's keys
dict.update(dict2)	Updates dictionary with specified key-value pairs
dict.values()	Returns a list of all the values of dictionary
pop()	Remove the element with specified key

<code>popItem()</code>	Removes the last inserted key-value pair
<code>dict.setdefault(key,default="None")</code>	set the key to the default value if the key is not specified in the dictionary
<code>dict.has_key(key)</code>	returns true if the dictionary contains the specified key.
<code>dict.get(key, default = "None")</code>	used to get the value specified for the passed key.

```
# demo for all dictionary methods
dict1 = {1: "Python", 2: "Java", 3: "Ruby", 4: "Scala"}
```

```
# copy() method
dict2 = dict1.copy()
print(dict2)
```

```
# clear() method
dict1.clear()
print(dict1)
```

```
# get() method
print(dict2.get(1))
```

```
# items() method
print(dict2.items())
```

```
# keys() method
print(dict2.keys())
```

```
# pop() method
dict2.pop(4)
print(dict2)
```

```
# popitem() method
dict2.popitem()
```

```
print(dict2)

# update() method
dict2.update({3: "Scala"})
print(dict2)

# values() method
print(dict2.values())
```

Output:

```
{1: 'Python', 2: 'Java', 3: 'Ruby', 4: 'Scala'}
{}
Python
dict_items([(1, 'Python'), (2, 'Java'), (3, 'Ruby'), (4, 'Scala')])
dict_keys([1, 2, 3, 4])
{1: 'Python', 2: 'Java', 3: 'Ruby'}
{1: 'Python', 2: 'Java'}
{1: 'Python', 2: 'Java', 3: 'Scala'}
dict_values(['Python', 'Java', 'Scala'])
```

UNIT – 4: Functions and Modules**Python User defined functions**

A function is a set of statements that take inputs, do some specific computation, and produce output. The idea is to put some commonly or repeatedly done tasks together and make a function so that instead of writing the same code again and again for different inputs, we can call the function. Functions that readily come with Python are called [built-in functions](#). Python provides built-in functions like [print\(\)](#), etc. but we can also create your own functions. These functions are known as **user defines functions**.

Table of Content

- [User defined functions](#)
- [Parameterized functions](#)
- [Function with return value](#)

Python User-defined functions

All the functions that are written by any of us come under the category of user-defined functions. Below are the steps for writing user-defined functions in [Python](#).

- In Python, a [def keyword](#) is used to declare user-defined functions.
- An indented block of statements follows the function name and arguments which contains the body of the function.

Syntax:

```
def                                     function_name():
    statements
    .
    .
```

Example: Here we have created the fun function and then called the fun() function to print the statement.

```
# Declaring a function
def fun():
    print("Inside function")

# Driver's code
# Calling function
fun()
```

Output:

Inside function

Python Parameterized Function

The function may take arguments(s) also called parameters as input within the opening and closing parentheses, just after the function name followed by a colon.

Syntax:

```
def                                     function_name(argument1,          argument2,          ...):
    statements
    .
```

Example:

A simple Python function to check whether x is even or odd.

```
def evenOdd( x ):
    if (x % 2 == 0):
        print("even")
    else:
        print("odd")

# Driver code
evenOdd(2)
evenOdd(3)
```

Output:

even
odd

Python Default arguments

A [default argument](#) is a parameter that assumes a default value if a value is not provided in the function call for that argument. The following example illustrates Default arguments.

Example: We call myFun() with the only argument.

```
# Python program to demonstrate
# default arguments
def myFun(x, y = 50):
    print("x: ", x)
    print("y: ", y)

# Driver code
myFun(10)
```

Output:

x:
y: 50

10

Python Function with return value

Sometimes we might need the result of the function to be used in further processes. Hence, a function should also return a value when it finishes its execution. This can be achieved by a return statement.

A [return statement](#) is used to end the execution of the function call and “returns” the result (value of the expression following the return keyword) to the caller. The statements after the return statements are not executed. If the return statement is without any expression, then the special value [None](#) is returned.

Syntax:

```
def                                     fun():
    statements
    .
    .
    return [expression]
```

Example:

Python program to demonstrate return statement.

```
def add(a, b):

    # returning sum of a and b
    return a + b

def is_true(a):

    # returning boolean of a
    return bool(a)
```

```
# calling function
res = add(2, 3)
print("Result of add function is {}".format(res))

res = is_true(2 < 5)
print("\nResult of is_true function is {}".format(res))
```

Output:

Result of add function is 5

Result of is_true function is True

Recursion in Python

The term [Recursion](#) can be defined as the process of defining something in terms of itself. In simple words, it is a process in which a function calls itself directly or indirectly.

Advantages of using recursion

- A complicated function can be split down into smaller sub-problems utilizing recursion.
- Sequence creation is simpler through recursion than utilizing any nested iteration.
- Recursive functions render the code look simple and effective.

Disadvantages of using recursion

- A lot of memory and time is taken through recursive calls which makes it expensive for use.
- Recursive functions are challenging to debug.
- The reasoning behind recursion can sometimes be tough to think through.

Syntax:

```
def func(): <--
    |
    | (recursive call)
    |
func() ----
```

Example 1: A Fibonacci sequence is the integer sequence of 0, 1, 1, 2, 3, 5, 8....

```
# Program to print the fibonacci series upto n_terms

# Recursive function
def recursive_fibonacci(n):
    if n <= 1:
        return n
    else:
        return(recursive_fibonacci(n-1) + recursive_fibonacci(n-2))

n_terms = 10
```

```
# check if the number of terms is valid
if n_terms <= 0:
    print("Invalid input ! Please input a positive value")
else:
    print("Fibonacci series:")
    for i in range(n_terms):
        print(recursive_fibonacci(i))
```

Output

Fibonacci series:

```
0
1
1
2
3
5
8
13
21
34
```

Standard Library: Built-in Functions : TYPE CONVERSION FUNCTIONS

int(a, base): This function converts **any data type to an integer**. ‘Base’ specifies the **base in which the string is** if the data type is a string.

float(): This function is used to convert **any data type to a floating-point number**.

```
# initializing string
s = "10010"

# printing string converting to int base 2
c = int(s,2)
print ("After converting to integer base 2 : ", end="")
print (c)

# printing string converting to float
e = float(s)
print ("After converting to float : ", end="")
print (e)
```

Output:

After converting to integer base 2 : 18
After converting to float : 10010.0

Python Type conversion using tuple(), set(), list()

tuple(): This function is used to convert to a tuple.

set(): This function returns the type after converting to set.

list(): This function is used to convert **any data type to a list type**.

```
# initializing string
s = 'geeks'

# printing string converting to tuple
c = tuple(s)
print ("After converting string to tuple : ",end="")
print (c)

# printing string converting to set
c = set(s)
print ("After converting string to set : ",end="")
print (c)

# printing string converting to list
c = list(s)
print ("After converting string to list : ",end="")
print (c)
```

Output:

After converting string to tuple : ('g', 'e', 'e', 'k', 's')
After converting string to set : {'k', 'e', 's', 'g'}
After converting string to list : ['g', 'e', 'e', 'k', 's']

Python code to demonstrate Type conversion using dict(), complex(), str()

dict(): This function is used to convert a tuple of order (key, value) into a dictionary.

str(): Used to convert an integer into a string.

complex(real,imag) : This function converts real numbers to complex(real,imag) number.

```
# initializing integers
a = 1
b = 2

# initializing tuple
tup = (('a', 1) ,('f', 2), ('g', 3))

# printing integer converting to complex number
c = complex(1,2)
print ("After converting integer to complex number : ",end="")
print (c)
```

```
# printing integer converting to string
c = str(a)
print ("After converting integer to string : ",end="")
print (c)

# printing tuple converting to expression dictionary
c = dict(tup)
print ("After converting tuple to dictionary : ",end="")
print (c)
```

Output:

```
After      converting      integer      to      complex      number      :      (1+2j)
After      converting      integer      to      string      :      1
After converting tuple to dictionary : {'a': 1, 'f': 2, 'g': 3}
```

Creating and Importing a module

A module is simply a Python file with a .py extension that can be imported inside another Python program. The name of the Python file becomes the module name. The module contains definitions and implementation of classes, variables, and functions that can be used inside another program.

Example: Let's create a simple module named GFG.

```
""" GFG.py """

# Python program to create
# a module


# Defining a function
def Geeks():
    print("GeeksforGeeks")


# Defining a variable
location = "Noida"
```

The above example shows the creation of a simple module named GFG as the name of the above Python file is GFG.py. When this code is executed it does nothing because the function created is not invoked.

To use the above created module, create a new Python file in the same directory and import GFG module using the import statement.

Example:

```
# Python program to demonstrate
```

```
# modules

import GFG

# Use the function created
GFG.Geeks()

# Print the variable declared
print(GFG.location)
```

Output:

GeeksforGeeks

Noida

The above example shows a simple module with only functions and variables. Now let's create a little bit complex module with classes, functions, and variables. Below is the implementation.

Example: Open the above created GFG module and make the following changes.

```
""" GFG.py """

# Python program to demonstrate
# modules

# Defining a function
def Geeks():
    print("GeeksforGeeks")

# Defining a variable
location = "Noida"

# Defining a class
class Employee():

    def __init__(self, name, position):
        self.name = name
        self.position = position

    def show(self):
        print("Employee name:", self.name)
        print("Employee position:", self.position)
```

In this example, a class named employee has been declared with a method show() to print the details of the employee. Now open the Python script for importing and using this module.

Example:

```
emp = GFG.Employee("Nikhil", "Developer")
emp.show()
```

Output:

Employee name: Nikhil
Employee position: Developer

Import all names

All the object from a module can be imported as a variable. This prevents the usage of the module name as a prefix.

Syntax:

```
from module_name_ import *
```

Example: We will use the above created GFG module.

```
Geeks()

# Printing the variable
print(location)

# Calling class
emp = Employee("Nikhil", "Developer")
emp.show()
```

Output:

GeeksforGeeks
Noida
Employee name: Nikhil
Employee position: Developer

Import with renaming

A module can be imported with another name, specified by the user.

Example:

```
import GFG as g
# Calling the function
g.Geeks()

# Printing the variable
print(g.location)
# Calling class
emp = g.Employee("Nikhil", "Developer")
emp.show()
```

Output:

GeeksforGeeks
Noida
Employee name: Nikhil
Employee position: Developer

Python datetime module

In Python, date and time are not data types of their own, but a module named **DateTime** in Python can be imported to work with the date as well as time. **Python Datetime module** comes built into Python, so there is no need to install it externally.

Get the Current Date

To return the current local date today() function of the date class is used. today() function comes with several attributes (year, month, and day). These can be printed individually.

```
# Python program to
# print current date

from datetime import date

# calling the today
# function of date class
today = date.today()

print("Today's date is", today)
```

Output

Today's date is 2021-08-19

Get Today's Year, Month, and Date

We can get the year, month, and date attributes from the date object using the year, month and date attribute of the date class.

```
from datetime import date

# date object of today's date
today = date.today()

print("Current year:", today.year)
print("Current month:", today.month)
print("Current day:", today.day)
```

Output

Current year: 2021
Current month: 8
Current day: 19

Get Date from Timestamp

We can create date objects from timestamps by using the `fromtimestamp()` method. The timestamp is the number of seconds from 1st January 1970 at UTC to a particular date.

```
from datetime import datetime

# Getting Datetime from timestamp
date_time = datetime.fromtimestamp(1887639468)
print("Datetime from timestamp:", date_time)
```

Output

Datetime from timestamp: 2029-10-25 16:17:48

Unit 5: String Processing and File Handling

Introduction to String

- A string is a sequence of characters.
- Strings can be created by enclosing characters inside a single quote or double-quotes.
- A string that contains no characters are called empty string.
- Python does not have a character data type, a single character is simply a string with a length of 1.

Example:

"Geeksforgeeks" or 'Geeksforgeeks' or "a"

```
print("A Computer Science portal for geeks")
print('A')
```

Output:

A Computer Science portal for geeks
A

Creating a String in Python

Strings in Python can be created using single quotes or double quotes or even triple quotes. Let us see how we can define a string in Python.

Example:

In this example, we will demonstrate different ways to create a Python String. We will create a string using single quotes (' '), double quotes (" "), and triple double quotes ("""" """). The triple quotes can be used to declare multiline strings in Python.

```
# Python Program for
# Creation of String

# Creating a String
# with single Quotes
String1 = 'Welcome to the Geeks World'
print("String with the use of Single Quotes: ")
print(String1)

# Creating a String
# with double Quotes
```

```
String1 = "I'm a Geek"
print("\nString with the use of Double Quotes: ")
print(String1)

# Creating a String
# with triple Quotes
String1 = "I'm a Geek and I live in a world of "Geeks""
print("\nString with the use of Triple Quotes: ")
print(String1)

# Creating String with triple
# Quotes allows multiple lines
String1 = "Geeks
          For
          Life"
print("\nCreating a multiline String: ")
print(String1)
```

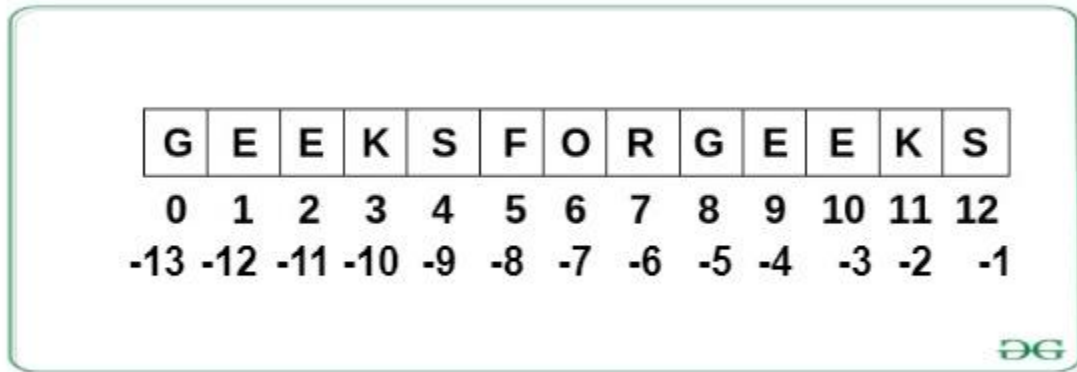
Output:

String	with	the	use	of	Single	Quotes:
Welcome		to	the		Geeks	World
String	with	the	use	of	Double	Quotes:
I'm			a			Geek
String	with	the	use	of	Triple	Quotes:
I'm	a	Geek	and	I	live	in
Creating		a			a	world
Geeks					multiline	of
						"Geeks"
						String:
	For					
	Life					

Accessing characters in Python String

In Python, individual characters of a String can be accessed by using the method of Indexing. Indexing allows negative address references to access characters from the back of the String, e.g. -1 refers to the last character, -2 refers to the second last character, and so on.

While accessing an index out of the range will cause an **IndexError**. Only Integers are allowed to be passed as an index, float or other types that will cause a **TypeError**.



Python String indexing

Example:

In this example, we will define a string in Python and access its characters using positive and negative indexing. The 0th element will be the first character of the string whereas the -1th element is the last character of the string.

```
# Python Program to Access
# characters of String

String1 = "GeeksForGeeks"
print("Initial String: ")
print(String1)

# Printing First character
print("\nFirst character of String is: ")
print(String1[0])

# Printing Last character
print("\nLast character of String is: ")
print(String1[-1])
```

Output:

```
Initial                                     String:
GeeksForGeeks
First           character           of           String           is:
G
Last           cha           racter           of           String           is:
s
```

String Slicing

In Python, the [String Slicing](#) method is used to access a range of characters in the String. Slicing in a String is done by using a Slicing operator, i.e., a colon (:). One thing to keep in mind while using this method is that the string returned after slicing includes the character at the start index but not the character at the last index.

Example:

In this example, we will use the string-slicing method to extract a substring of the original string. The `[3:12]` indicates that the string slicing will start from the 3rd index of the string to the 12th index, (12th character not including). We can also use negative indexing in string slicing.

```
# Python Program to
# demonstrate String slicing

# Creating a String
String1 = "GeeksForGeeks"
print("Initial String: ")
print(String1)

# Printing 3rd to 12th character
print("\nSlicing characters from 3-12: ")
print(String1[3:12])

# Printing characters between
# 3rd and 2nd last character
print("\nSlicing characters between " +
      "3rd and 2nd last character: ")
print(String1[3:-2])
```

Output:

Initial								String:
GeeksForGeeks								
Slicing	characters				from			3-12:
ksForGeek								
Slicing	characters	between	3rd	and	2nd	last		character:
ksForGee								

Reversing a Python String

By accessing characters from a string, we can also [reverse strings in Python](#). We can Reverse a string by using String slicing method.

Example:

In this example, we will reverse a string by accessing the index. We did not specify the first two parts of the slice indicating that we are considering the whole string, from the start index to the last index.

```
#Program to reverse a string
gfg = "geeksforgeeks"
print(gfg[::-1])
```

Output:

skeegrofskeeg

Example:

We can also reverse a string by using built-in [join](#) and [reversed](#) functions, and passing the string as the parameter to the reversed() function.

```
# Program to reverse a string

gfg = "geeksforgeeks"

# Reverse the string using reversed and join function
gfg = "".join(reversed(gfg))

print(gfg)
```

Output:

Skeegrofskeeg

Deleting a character

Python strings are immutable, that means we cannot delete a character from it. When we try to delete the character using the **del** keyword, it will generate an error.

```
# Python Program to delete
# character of a String

String1 = "Hello, I'm a Geek"
print("Initial String: ")
print(String1)

print("Deleting character at 2nd Index: ")
del String1[2]
print(String1)
```

Output:

Initial								String:
Hello,		I'm		a				Geek
Deleting		character		at		2nd		Index:
Traceback		(most		recent		call		last):
File	"e:\GFG\Python	codes\Codes\demo.py",		line	9,	in		<module>
del								String1[2]

TypeError: 'str' object doesn't support item deletion

But using slicing we can remove the character from the original string and store the result in a new string.

Example:

In this example, we will first slice the string up to the character that we want to delete and then concatenate the remaining string next from the deleted character.

```
# Python Program to Delete
# characters from a String

String1 = "Hello, I'm a Geek"
print("Initial String: ")
```

```

print(String1)

# Deleting a character
# of the String
String2 = String1[0:2] + String1[3:]
print("\nDeleting character at 2nd Index: ")
print(String2)

```

Output:

Initial						String:
Hello,		I'm		a		Geek
Deleting	character		at		2nd	Index:
Helo, I'm a Geek						

Deleting Entire String

Deletion of the entire string is possible with the use of del keyword. Further, if we try to print the string, this will produce an error because the String is deleted and is unavailable to be printed.

```

# Python Program to Delete
# entire String

String1 = "Hello, I'm a Geek"
print("Initial String: ")
print(String1)

# Deleting a String
# with the use of del
del String1
print("\nDeleting entire String: ")
print(String1)

```

- **MAX (string)** returns the character with the highest Unicode value in the string.

Str = "zebracrossing"

Print(max(str))

O/P: z

- **MIN (string)** returns the character with the lowest Unicode value in the string.

```
Str = "zebracrossing"  
Print(min(str))  
O/P: a
```

- **string.isnumeric ()** returns True if all the characters in the string are numeric, such as digits, subscripts, superscripts, fractions, etc.

- `"-1".isnumeric()`
- `False`

- **string.isalpha ()** returns True if all the characters in the string are alphabetic, such as letters, accented characters, etc.

```
STR = "12345"  
PRINT(STR.ISALPHA())  
O/P: TRUE
```

- **string.isdigit ()** returns True if all the characters in the string are digits, such as 0-9, subscripts, superscripts, etc.

- `"\u00B2".isdigit() # superscript 2`
- `True`

- This classification still does not work for negative numbers:

- `>>> "-1".isdigit()`
- `False`

- **string.isidentifier ()** returns True if the string is a valid i

entifier, meaning it follows the rules for naming variables, functions, classes, etc. in Python.

- `>>> "hello".isidentifier()`
- `True`
- `>>> "1hello".isidentifier()`
- `False`

- `>>> "h_e_l_l_o".isidentifier()`
- `True`

- **string.islower ()** returns True if all the cased characters in the string are lowercase.

Str = "india"

Print(str.islower())

O/P: TRUE

- **string.isupper ()** returns True if all the cased characters in the string are uppercase.

Str = "INDIA"

Print(str.isupper())

O/P: True

- **string.isspace ()** returns True if all the characters in the string are whitespace, such as spaces, tabs, newlines, etc.

- `" ".isspace() # four spaces`
- `True`
- `>>> "\t".isspace() # a tab`
- `True`
- `>>> "\n"`
- `...`
- `... "\n".isspace() # a multiline string`
- `True`